

Development of Battlemage Arena

1st Toke Odgaard-Jans
toodg22@student.sdu.dk

2nd Marcus Elmkvist
maelm22@student.sdu.dk

3rd Oliver
owagn20@student.sdu.dk

4th Mansoor
moahm25@student.sdu.dk

I. INTRODUCTION

The goal of this project is to create a multiplayer game centered around physics-based interactions. The game takes place in a magical tournament where two players compete in an arena using physics-driven spells.

Unlike traditional combat systems, players cannot directly damage each other. Instead, damage occurs indirectly through environmental interactions. The damage calculation is based on physical forces, heavier and faster objects cause more damage.

Spells were supposed to interact dynamically with the environment, allowing creative combinations for unique effects. Players were also supposed to take damage from traps or hazards within the arena. To enhance realism and fun, ragdoll physics is implemented for player characters.

II. GAME DESIGN

A. Overview

Battle Mage Arena is a multiplayer game where players control mages in an arena and use physics-based spells to manipulate the environment. The core mechanic is indirect damage: players cannot harm each other directly but must use spells to create environmental hazards that deal damage through physical interactions.

B. Main Mechanics

- **Physics-Based Combat:** Spells do not deal direct damage. Instead, they apply forces to objects, creating dynamic interactions. Damage is calculated based on impact force using the simplified formula:

$$\text{Force} = m \times v \quad (1)$$

Where m is the object's mass and v its velocity. Heavier and faster objects cause more damage.

- **Spells:**
 - **Force Spell:** Applies an impulse to objects via a raycast, pushing them in the opposite direction of the cast.
 - **Ice Spell:** Creates an ice surface that removes friction for actors in contact, enabling creative movement and trap setups.
 - **Mass Increase Spell:** Upon hitting a synchronized object with a raycast, the objects mass and size are both increased.
- **Health and Ragdoll:** When a player is hit with sufficient force, they take damage and enter ragdoll mode for a short cooldown period.

- **Multiplayer Synchronization:** The game uses Unreal Engine's listen server model. Positions, velocities, and spell effects are synchronized across clients to maintain consistent physics interactions.
- **Game Modes:** Multiple game modes are supported through a parent-child blueprint structure, allowing easy extension and customization.
- **HUD and Feedback:** Players have a HUD displaying health, score, and a timer. Audio feedback includes spell sounds.
- **Levels and level Selector:** The games has three levels
 - **Arena Battle:** A small arena filled with synchronized physics boxes. The goal is to eliminate the other player by hitting them with a box.
 - **Soccer:** A small soccer arena with a ball in the middle, the goal is to get the ball into the other players' goal. First to 5 wins.
 - **Coin Collection:** Small arena filled with synchronized physics boxes. Coins spawn at random points within the level. The goal is to collect 5 coins to win. If hit by a physics box you will lose coins.

III. IMPLEMENTATION

A. Damage

The damage implementation was twofold. First, it involved a parent blueprint, "BP_Synced_Box" designed for all objects that could be affected by forces and for damage detection on player characters. Once a hit is detected on an object, it checks whether the collided object is a player blueprint. If this is the case, it calls the "Hitting" event on the character blueprint and passes in itself and the component that detected the collision (see Figure 1).

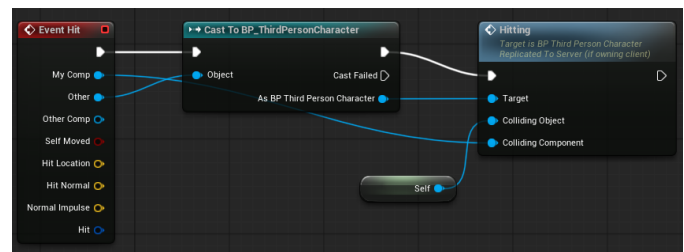


Fig. 1. Hit Detection Blueprint which calls the hitting event on the player character blueprint.

In the "Hitting" event, it is checked whether the collision was detected on the server or the client. If the detection occurs

on the client, nothing happens; however, if it occurs on the server, the process continues. It then checks whether the player can currently receive any damage. If so, it proceeds. As a precaution, it verifies that the event is being called by a synced object that is supposed to call it (see Figure 2). If this is the case, the process continues for the damage calculation.

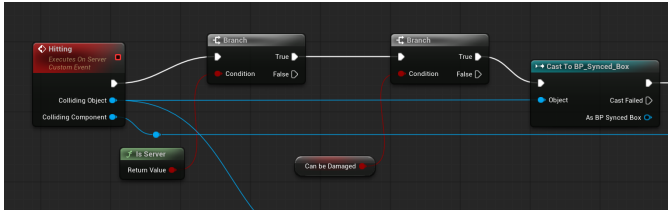


Fig. 2. Tjeks whether the hit was detected on the server or on a client.

From the provided references, both the mass of the colliding object and its velocity at the moment of collision are extracted. They are then multiplied together to calculate the force with which the player is hit. The force is checked to see if it exceeds the minimum threshold of 50. If it does, the force is clamped between specified minimum and maximum values and mapped to corresponding minimum and maximum damage values. This damage is then applied to the player's health through a function call, Take Damage (see Figure 3).

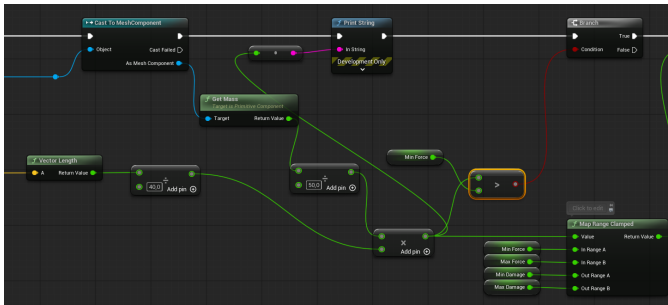


Fig. 3. Calculating damage based on velocity and mass.

B. Ragdoll

C. Parent Spell Blueprint

A parent spell blueprint is created to a standard way to "fire" a spell. So when spells are created by different people the way to cast them is the same. The event in the parent blueprint is called "Activate Spell".

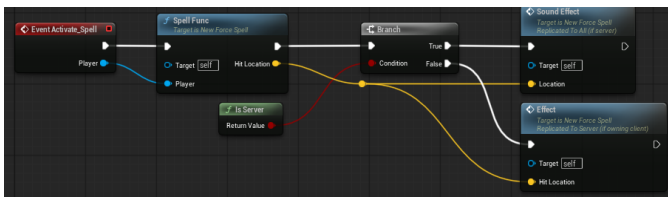


Fig. 4. Implementation of "Activate Spell" in the Force Spell child blueprint.

The parent event "Activate Spell" is being used in the Force Spell child blueprint. When the event is called, the spell logic

is executed along with the different spell effects. The player as an input makes possible to get any details about the player, that are necessary for the spell. And if a spell requires something new, it can be added to the player, this could be a new location to fire a spell from. That is possible with having to change the parent blueprint.

1) *Force Push*: How the "Force Push" spell works is that when the player casts the spell, raycasts are shot in the direction the player is looking. There are "Spell points" which just are static meshes with no meshes, they are just points and they are places at the camera. Then the raycasts are shot from these points, they form a cylinder shape making it easier to hit objects. When these raycasts hit a "BPSyncedBox" object, the "Apply Force" function is called on the hit object to make it move.

2) Ice Spell:

3) *Mass Increase*: When the growth spell is cast, we set an increment factor (IncFactor) to 1.15, corresponding to a 15% increase in size and mass. We then perform a raycast (line trace), similar to the one used for the Force spell, and read the resulting HitResult. If the hit actor is of type BP_Synced_Box, we first access its Static Mesh Component and obtain its current mass. This mass is multiplied by IncFactor, and the new value is applied using the Set Mass Override function. Next, we retrieve the actor's root scene component, read its current world scale, multiply each axis of the scale by IncFactor, and write the result back using Set World Scale 3D. Consequently, each successful cast of the spell makes the targeted box uniformly larger and heavier.

D. Healthbar

The health bar system is implemented using Unreal Engine's replicated variables and widget components to ensure consistent feedback in a multiplayer environment. Each character owns a CurrHealth variable that is replicated from the server to all clients using a RepNotify function, OnRep_CurrHealth. The server is authoritative for all health changes: whenever damage or healing occurs, the server updates CurrHealth. When this updated value is received on a client, Unreal automatically invokes OnRep_CurrHealth for the corresponding character instance on that machine.

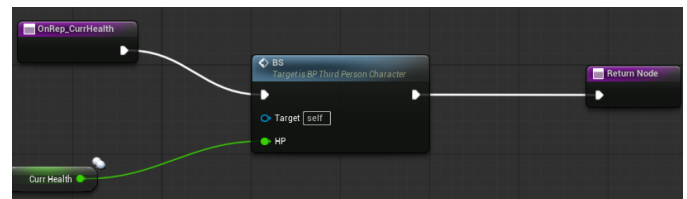


Fig. 5. OnRep Function

OnRep_CurrHealth triggers a custom event (named BS), which is responsible for driving the visual update of the health bar. This event accesses the character's HealthBar widget component, retrieves its underlying user widget, and casts it to the dedicated HUD widget blueprint (WBP_HUD). The event

then calls the widget's UpdateHealthBar function, passing both the new health value and a visibility flag derived from IsLocallyControlled, so that a player does not see a bar above their own character but can see bars above other players.

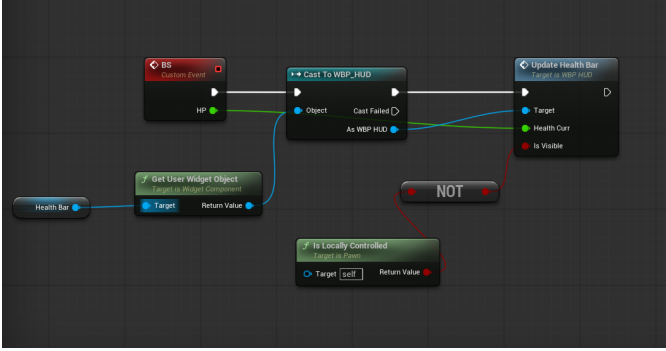


Fig. 6. The BS event called by OnRep_CurrHealth to update the health bar widget.

Inside WBP_HUD, UpdateHealthBar converts the raw health value to a 0–1 range, applies it to the progress bar via SetPercent, and adjusts the widget's visibility accordingly. This design cleanly separates gameplay logic from UI presentation while guaranteeing that all clients display accurate, up-to-date health information. Since, the owning player cannot see their own health bar, we create simple healthbar widget and add it to the viewport in the player controller when the game starts.

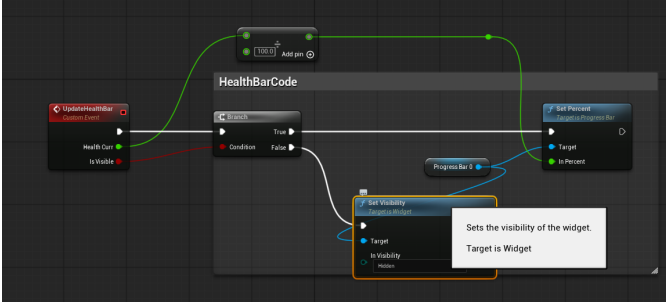


Fig. 7. UpdateHealthBar Widget

E. Spell CoolDown

The spell cooldown system is implemented as a timestamp-based mechanism in the character, combined with a generic cooldown-bar widget for visual feedback.

Character-side Logic: For each spell the character stores two floating-point variables:

- Force Spell Interval, Mass Inc Spell Interval, Ice Spell Interval: the cooldown duration in seconds.
- Force Spell CLDWN, Mass Inc Spell CLDWN, Ice Spell CLDWN: the game time at which that spell was last successfully cast.

In BeginPlay, all ... CLDWN values are initialised to 0.0 (meaning “not used yet”), and the interval values are set

(e.g. Force = 0.4 s, Ice = 2.0 s). At the same time, a cooldown widget instance is created for each spell using Create Spell Cooldown Bar Widget. These instances are stored in variables such as Force Spell CLDWNWidget, added to the viewport, and then hidden. Only the widget corresponding to the currently selected spell is set to Visible.

When the player presses Fire, the character first reads the Spell integer from ThirdPersonPlayerController to determine which spell is active. For that spell it computes

$$\text{Elapsed} = \text{GetGameTimeSinceCreation}(\text{self}) - \text{SpellCLDWN}.$$

Elapsed is then compared with the corresponding interval (Spell Interval). If

$$\text{Elapsed} \geq \text{Interval},$$

the spell is deemed off cooldown and Activate Spell (a server RPC) is called. Immediately afterwards, SpellCLDWN is updated to the current value of GetGameTimeSinceCreation, effectively starting a new cooldown, and that spell's cooldown widget is made visible.

Widget-side Logic: The SpellCooldown widget maintains its own Time and MaxTime variables. On Event Tick, it runs CountdownBarTimer, which increments Time by DeltaSeconds, clamps it to the interval [0, MaxTime], and normalises it to the 0–1 range. This normalised value is applied to the progress bar via SetPercent, visually filling the bar as the cooldown elapses. A Delay node using MaxTime then hides the widget once the cooldown duration finishes.

In summary, the character's timestamp-based variables enforce the actual gameplay cooldown, while the dedicated widget provides an independent, frame-based visual countdown that reflects the same duration.

F. Game Modes

There are three different game modes in the game, Arena Battle, Soccer, and Coin Collection. A gamemode consists of two blueprints, a gamemode and a gamestate blueprint. A parent gamemode and gamestate blueprint is made, and each gamemode has its own child gamemode and gamestate blueprint. This is to centralize the shared logic and variables between gamemodes and reduce code duplication. An example of this is the function “Initial Player setup”, this function adds values to the variables “Player 1” and “Player 2” of the Character type. And adds them to the array “Players”. It also sets the player location to that of their spawn points. This function is called in all the gamemodes’ “Begin Play” event, so the code is not duplicated in each gamemode. After “Initial Player Setup” is called, the gamemode specific logic begins. For example in the Coin Collection gamemode, the spawn points for the coins are collected and put into the array “Coin Spawn Points”, and then spawns a coin. It will then set the players scores to 0. The parent gamestate only does one thing, it has a int variable called “Gamemode int”, and this variable is set by the children states. This is to tell the HUD which gamemode is currently being played so it can update

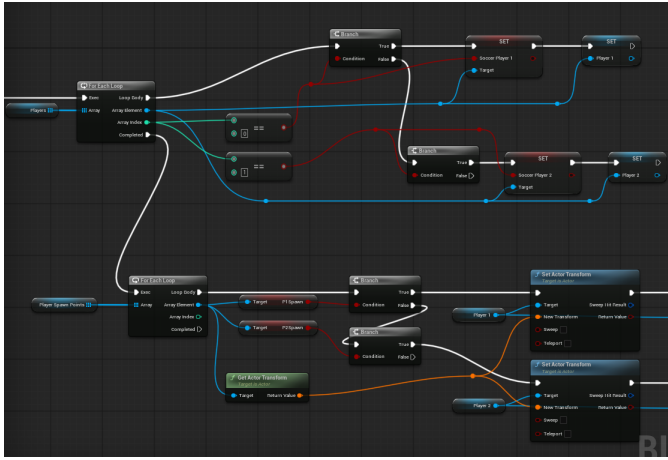


Fig. 8. A Part of the "Initial Player Setup" function in the parent gamemode.

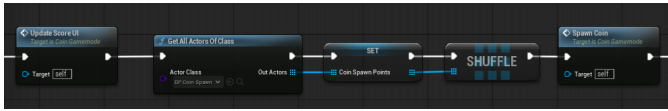


Fig. 9. What comes after "Initial Player Setup" in the Coin Collection gamemode.

the display accordingly. It is also used to enable gamemode specific features in the player character blueprint, depending on the "gamemode int" value the healthbar will show. The reason to have the "gamemode int" variable in the gamestate is that gamestates are replicated to clients, and gamemodes are not. So a client, not the hosting client which is also the server, would not be able to see the "gamemode int" if it were in the gamemode.

G. HUD

The HUD that is displayed to the player is made using the UserWidget blueprint. Multiple are being used at the same time to display the different elements of the HUD. And with multiple gamemodes, some UI elements are only necessary for certain gamemodes. So the approach to making the HUD was to be modular, and make small widgets that only handled one thing. Then in the Player Controller called BPThirdPersonPlayerController, in the function called "UI Setup" was made. In "UI Setup" all the widgets are created and added to the viewport depending on the gamemode. How this is achieved is by getting the gamemode int from the gamestate, and match that with the correct branch. At the correct branch, the widgets are created and added to the viewport. This makes the HUD easy to change and add too.

1) *Spell Selector*: The spell selector widget is the user widget that displays the spells available to the player, and the current one selected. The widget displays three images of the same color, the color is different if the spell is selected or not. The "change select" function is called when the player uses the scroll wheel to change the spell. The function takes

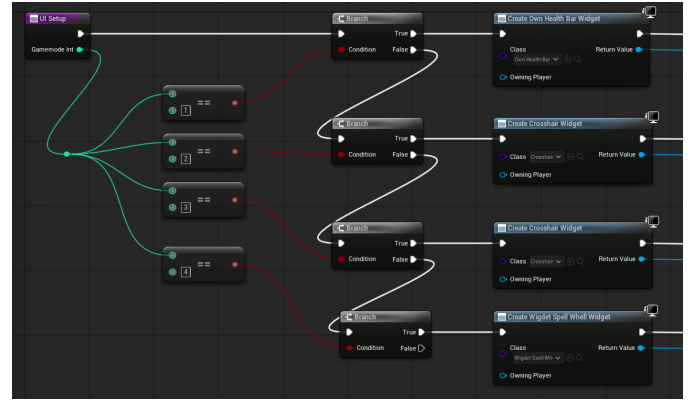


Fig. 10. UI Setup in the Player Controller.

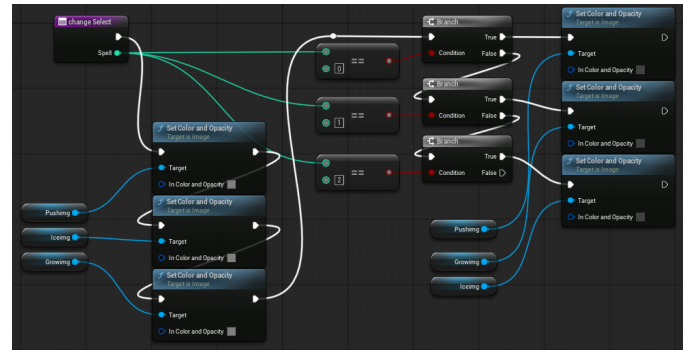


Fig. 11. The "change select" function.

an integer as an input, and the function will change the color of the corresponding spell to the integer.

IV. DEVELOPMENT

Xxxxx

A. Iteration 1

The first iteration was focussed on setting up the project and implementing some basic online functionality.

1) *About*: The purpose of this iteration was to setup the project and implement basic functionality for testing purposes. We needed to install Unreal Engine, ensure all the team members could work on the project, and set up version control using git. Character movement and physics manipulation should be synchronized across client and server. As such, this iteration also concerned learning about the Unreal Engine 5 client/server model, synchronizing player movement, and implementing a physics object which had its location and physics properties synchronized across client and server.

We also implemented a force spell that pushes the object in a direction by applying a force to it.

Finally, we implemented a simple health system for the player, so that colliding with the box would result in the player taking damage.

2) *Evaluation*: Since the goal of this iteration was to test if the technical aspects of the game worked, testing was done internally in the group. After a feature had been developed, it would be playtested on two connected clients that were simulated by the engine. The success of this iteration was judged to the extent that the synchronized objects behaved as expected.

We used the built-in systems for synchronization and online play to synchronize the box, the player, and player health. Making it so the player took damage when hit by the box was more challenging. The game only registered a hit when the player walked into the box, not when the box was thrown at the player.

3) *Reflections and decisions*: Most systems worked well. For the next iteration it was agreed that we would get the player to take damage when hit by physics-affected objects. Otherwise, the next iteration would be focussed on adding new features to the game.

B. Iteration 2

This iteration concerned the development of more mechanics and ensuring that it functioned well to get it ready for user testing.

1) *About*: Xxxxx

2) *Evaluation*: Xxxxx

3) *Reflections and decisions*: Xxxxx

C. Iteration 3

Xxxxx

1) *About*: Xxxxx

2) *Evaluation*: Xxxxx

3) *Reflections and decisions*: Xxxxx

D. Final iteration

Xxxxx

3) **Expanded Spell Variety**: Currently, the game only features three spells, limiting the strategic interactions between players. By adding more spells, we could create opportunities for more complex interactions. For example, spells could be designed to spawn additional projectiles for players to use against opponents or create new environmental hazards. If fall damage were to be introduced, a spell that propels an opponent into the air could create thrilling moments in the game.

4) **Improved Controls and Movement Mechanics**: At present, player movement is limited to basic walking and jumping. Particularly in the soccer game mode, a running mechanic would significantly enhance player control and positioning, allowing for a more immersive gameplay experience.

Incorporating these mechanics would not only improve the visual and strategic elements of the game but also elevate the user experience by making the gameplay more engaging and interactive.

V. REFLECTION

Throughout this project, we have successfully developed the necessary mechanics and functionalities to effectively showcase the concept of the game. However, with additional time, there are several mechanics we would consider implementing to enhance gameplay and player experience.

1) **Environmental Hazards and Enhanced Maps**: The current game maps are somewhat simplistic in both aesthetics and functionality. Given more time for development, we could introduce environmental hazards that create dynamic challenges for players, enhancing the overall gameplay experience.

2) **Destruction of the Environment**: Enhancing the environment's interactivity through destructible elements would add depth. For instance, when a player uses a spell or interacts with an object (like a pillar), the environment could react by collapsing, with debris functioning as synchronized damage objects. This would not only increase visual appeal but also introduce strategic elements to gameplay as players navigate the altered terrain.